SASDenSebLE: A Compact Vision Transformer Inference Architecture With Saturation-Approximate Softmax Dataflow Enabling Sequence-Parallelism Boosted Layer-Fusion Execution

Liu He, *Graduate Student Member, IEEE*, Yujin Wang, *Graduate Student Member, IEEE*, Zongle Huang, *Graduate Student Member, IEEE*, Shupei Fan, *Graduate Student Member, IEEE*, Chen Tang, *Graduate Student Member, IEEE*, Shuyuan Zhang, *Graduate Student Member, IEEE*, Luchang Lei, *Graduate Student Member, IEEE*, Huazhong Yang, *Fellow, IEEE*, Yongpan Liu, *Senior Member, IEEE*, and Hongyang Jia, *Member, IEEE*

***Abstract*—Transformer-based neural networks (NNs) have been dominant in modern AI due to their superior accuracy and generalization over traditional models. However, the quadratic complexity and dataflow challenges of self-attention hinder their efficient deployment on edge devices, especially for latency-sensitive, single-batch inference. Limited data parallelism necessitates both pipeline and tensor parallelism. Specifically, sequence parallelism offers strong scaling potential in domain-specific accelerators (DSAs), but is constrained by the temporal dependencies involved in the max-finding operations of softmax. This work presents SASDenSebLE, a compact, scalable vision Transformer (ViT) inference architecture co-designed with a max-finding-free softmax approximation across micro-architecture, architecture, and algorithm levels. A dense-sparse-hybrid datapath transforms large-value softmax operations into linear projections, enabling layer-fusion. A hierarchical system further improves sequence parallelism without excessive on-chip buffering or off-chip traffic. Experiments show up to 2.83×, 28.02×, and 1.40× speedup over baseline DSAs and a state-of-the-art ViT accelerator design, respectively, with negligible accuracy loss. Demonstration on the camera encoder of a recent bird's-eye-view model shows up to 10.82× speedup over a typical edge GPU.**

***Index Terms*—Approximate softmax, scalability, SW-HW co-design, Transformer.**

Received 23 December 2024; revised 21 March 2025; accepted 5 April 2025. Date of publication 16 April 2025; date of current version 22 October 2025. This work was supported in part by the National Natural Science Foundation of China under Grant 62374101. This article was recommended by Associate Editor N. K. Jha. *(Corresponding author: Hongyang Jia.)*

Liu He, Yujin Wang, Zongle Huang, Shupei Fan, Chen Tang, Shuyuan Zhang, and Luchang Lei are with the Department of Electronic Engineering, Tsinghua University, Beijing 100084, China (e-mail:he-l23@mails.tsinghua.edu.cn; yujin-wa24@mails.tsinghua.edu.cn; huangzl23@mails.tsinghua.edu.cn; fansp19@mails.tsinghua.edu.cn; tangc20@mails.tsinghua.edu.cn; shuyuan-23@mails.tsinghua.edu.cn; llc22@mails.tsinghua.edu.cn).

Huazhong Yang, Yongpan Liu, and Hongyang Jia are with the Department of Electronic Engineering, Beijing National Research Center for Information Science and Technology, Tsinghua University, Beijing 100084, China (e-mail: yanghz@tsinghua.edu.cn; ypliu@tsinghua.edu.cn; hjia@tsinghua.edu.cn).

Digital Object Identifier 10.1109/TCAD.2025.3561719

I. Introduction

TRANSFORMER-BASED neural networks (NNs) have become essential in AI applications, including autonomous driving [1], [2], NLP [3], [4], and CV [5], [6], [7], [8]. Despite their exceptional performance, Transformer-based NNs entail significant computational and storage demands, imposing substantial challenges for both training and inference. The giant Transformer NNs are usually trained and fine-tuned on the cloud server for tens to thousands of hours before deploying to inference engines, either in the cloud or on the edge. Efficient Transformer inference is a key concern for edge and edge-server computing platforms. However, the state-of-the-art general-purpose edge-server compute devices have been pushed to their limit for executing Transformer NNs at the required speed, necessitating customized architecture design [9], [10], [11], [12], [13].

Efficient Transformer inference on edge devices faces critical challenges due to strict real-world constraints. In contrast to cloud inference, which can leverage input batching and larger buffering resources to significantly enhance model reuse, edge inference often involves single-batch inputs and prioritizes low latency, as required in tasks like BEV perception for autonomous driving [2]. The lack of batch-level parallelism highlights the need for alternative strategies, such as pipeline and tensor parallelism. Among these, sequence parallelism presents a promising avenue for scalable inference, complementing parallelism in the attention-head dimension, as illustrated in Fig. 1.

However, the inherent dataflow of self-attention the key innovation behind Transformers presents significant challenges in leveraging sequence parallelism. These difficulties primarily arise from the max-finding step in the softmax operation, which requires buffering entire attention matrix rows proportional to sequence length. In sequence-parallel execution, retaining multiple rows of the attention matrix exacerbates the buffering demands, further complicating efficient implementation. Local modifications in the self-attention datapath alone give a limited solution. Instead, a holistic rethinking of

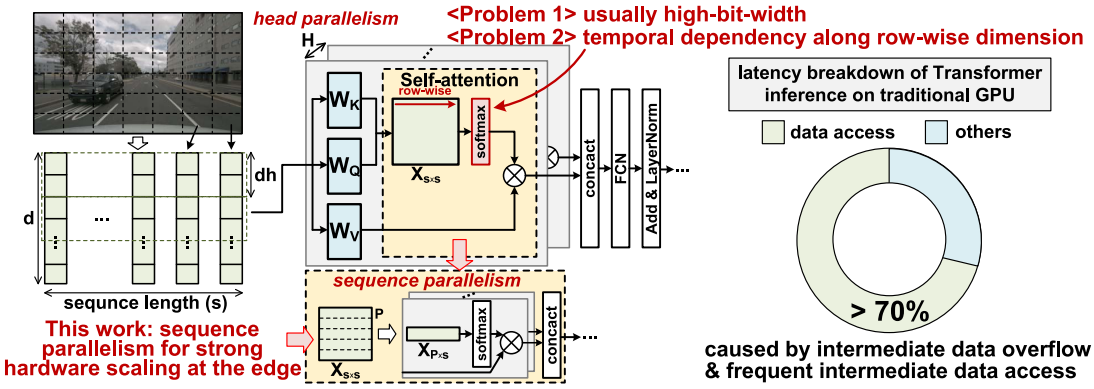


Fig. 1. Illustrations of head parallelism and sequence parallelism in Transformer inference. In traditional GPU, sequence parallelism suffers inefficiency due to frequent intermediate data movement.

self-attention dataflow, core-level architecture, and hierarchical memory systems is required to harness the potential gains from sequence parallelism.

Numerous pioneering works [14], [15], [16], [17] have concentrated on accelerating the nonlinear softmax computation. For instance, leveraging the principles of Taylor expansion, [15] uses a Taylor-based piecewise-linear LUT approximation, while [14], [17] combines max-finding and denominator accumulation in one pass to reduce memory access, though still requiring full row buffering. These approaches primarily focus on optimizing the softmax function itself, often overlooking the substantial memory access overhead and hardware storage demands that arise when scaling systems to accommodate large sequence lengths. FlashAttention [18], designed for GPU platforms, addresses this limitation by decoupling the temporal dependencies in softmax. This is achieved through iterative max-value updates and recomputation of softmax denominators. Despite these advantages, FlashAttention introduces recomputation overhead and relies on high-bit-width attention computations, which are challenging to directly implement in domain-specific accelerators (DSAs).

To achieve scalable and low-latency Transformer inference on edge devices, SASDenSebLE, an end-to-end architecture enabling fine-grained sequence parallelism co-optimized with approximate softmax dataflow is proposed. Rather than solely focusing on simplifying softmax computation, SASDenSebLE aims to minimize the buffering demands of the attention matrix and associated memory access overhead, thereby significantly enhancing system-level compute density. SASDenSebLE makes the following contributions

1) At the algorithmic level, SASDenSebLE employs a linear-after-saturation approximate softmax scheme, effectively eliminating the max-finding step and enabling fully low-bit-width self-attention execution with minimal accuracy degradation.
2) At the micro-architecture level, SASDenSebLE introduces a hybrid dense-sparse Transformer-processing element (TrPE) tailored for the linear approximate dataflow, which facilitates layer fusion between matrix multiplication (MM) layers in self-attention and distributed low-bit-width softmax evaluation, significantly reducing memory access requirements for the attention matrix.
3) At the system architecture level, SASDenSebLE presents an end-to-end design that leverages fine-grained sequence parallelism while minimizing buffering requirements for the attention matrix, achieving efficient utilization of on-chip memory and enabling low-latency inference on the edge.
4) SASDenSebLE achieves up to a 10.82× speedup compared to the NVIDIA Jetson AGX Orin GPU [19] when executing the camera encoder in recent BEV tasks [20]. When compared to the state-of-the-art DSA design [13], SASDenSebLE delivers up to 2.83× and 28.02× speedups, with energy efficiency improvements of 1.31× and 3.87×, respectively, across two baseline configurations. Additionally, compared to the Vision Transformer (ViT) accelerator ViTCoD [21], SASDenSebLE achieves a maximum speedup of 1.4×.

II. Background and Motivations

A typical Transformer encoder block comprises self-attention and linear projection layers. Self-attention involves two MM layers and a softmax layer, with Q, K, and V matrices generated by preceding linear layers. Specifically, $Q \cdot K^T$ reduces along the embedding dimension, producing an attention matrix X that scales quadratically with sequence length. After applying softmax to X to obtain attention scores A, A is multiplied by V to produce the output O. A standard softmax operation is denoted in (1), where s represents the sequence length

$$\text{Softmax}(X_i) = \left[\frac{e^{x_{i,j}-x_{i,\max}}}{\sum_{j=1}^{s} e^{x_{i,j}-x_{i,\max}}}, \ldots, \right]^T, j = 1, \ldots, s. \quad (1)$$

In addition to its computational complexity, the softmax operation poses two challenges for efficient edge inference of Transformer NNs: 1) excessive intermediate data movement and 2) impractical attention matrix buffering when exploiting sequence parallelism.

Algorithm 1 Standard Softmax on Vector X_i

1: $m_{\max} \leftarrow x_{i,1}$
2: denominator $\leftarrow 0$
3: **for** $j \leftarrow 1, s$ **do** **Step1**
4: $\quad m_{\max} \leftarrow \max(m_{\max}, x_{i,j})$
5: **end for**
6: **for** $j \leftarrow 1, s$ **do** **Step2**
7: $\quad$ denominator $+= \exp(x_{i,j} - x_{i,\max})$
8: $\quad y_{i,j} \leftarrow \exp(x_{i,j} - x_{i,\max})$
9: **end for**
10: **for** $j \leftarrow 1, s$ **do** **Step3**
11: $\quad y_{i,j} \leftarrow \frac{y_{i,j}}{\text{denominator}}$
12: **end for**

A. Intermediate Data Movement in Softmax

The first bottleneck arises from core-level data movement. Standard softmax requires computing $x_{i,\max}$ by scanning the entire attention row X_i, which exceeds on-chip buffer capacity for long sequences when sequence parallelism is applied (Section II-B), forcing data spills to higher memory levels. The multiple read and write operations for intermediate data exacerbate the data movement overhead: First, reading all elements of the large-dimensional X_i to identify the maximum value (Algorithm 1 Step 1). Second, computing and accumulating exponents in the denominator for normalization requires a read and write to memory (Algorithm 1 Step 2). Finally, each exponent undergoes normalization, involving a read of the exponent values and a write of the final softmax results (Algorithm 1 Step 3). This requires three reads and two writes from higher-level memory with high bit-width, while only performing four calculations (maximum subtraction, exponentiation, denominator accumulation, and normalization). The low arithmetic intensity of 0.8 results in a memory-bound computation, even if assuming buffering on-chip. This necessitates low-bit-width softmax computation integrated with the fusion of MM layers in self-attention, effectively mitigating the overhead of excessive intermediate attention matrix access.

B. On-Chip Buffering for Sequence Parallelism

Sequence parallelism leverages interrow independence in the attention matrix to scale beyond head parallelism. Fig. 2(a) shows how one attention head is partitioned under sequence parallelism, where each core computes submatrices, e.g., $Q_{\text{x-y}} \cdot K^T = X_{\text{x-y}}$ and $A_{\text{x-y}} \cdot V = O_{\text{x-y}}$. Assuming there are N cores for the calculation of one attention head and sequence parallelism is implemented among these cores. Denoting the partition size of Q matrix as $P \cdot dh$ and the degree of sequence parallelism as N, each time N partitions are computed spatially, requiring T rounds temporally to complete the calculation of an entire attention head. It is important to note that the temporal dependency, namely, the max-finding operation introduced by the standard softmax necessitates that $A \cdot V$ must wait for the complete rows of the attention matrix to be ready for processing. The buffering requirement of the intermediate attention matrix is $P \cdot s \cdot N$, which is proportional to both the sequence length and the degree of sequence parallelism. As shown in Fig. 2(b) and (c), this buffering

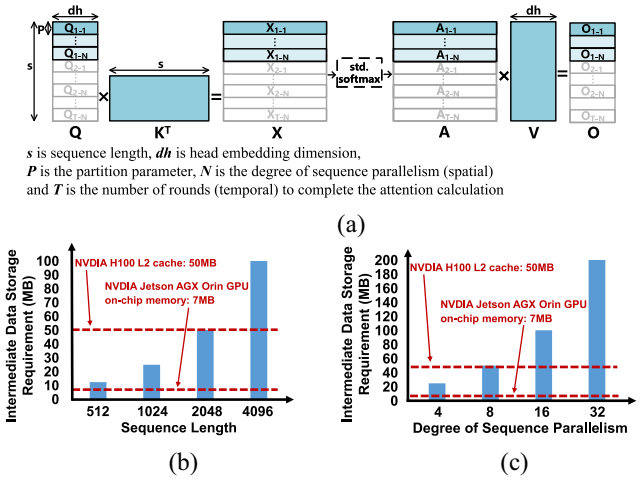


Fig. 2. (a) Matrix partition for sequence parallelism in attention module. The intermediate data storage requirement is proportional to (b) sequence length (with N fixed to 8) (c) degree of sequence parallelism (with s fixed to 1024). Here assume the intermediate data width is 32-bit and a head parallelism of 12.

can exceed typical on-chip memory limits on GPUs (e.g., H100, Jetson AGX Orin) in long-sequence, high-sequence-parallelism settings.

Storing intermediate data in off-chip DRAM, which is bandwidth-limited and energy-inefficient, underutilizes on-chip Transformer compute resources. Expanding on-chip SRAM to avoid off-chip access is impractical, as buffering scales with sequence length and parallelism, leading to excessive area overhead and reduced compute density. This creates a dilemma between achieving high sequence parallelism for strong scaling and maintaining a feasible hardware configuration on the edge, a challenge primarily driven by the max-finding step in the standard softmax operation. The issue worsens at advanced nodes where SRAM scaling slows, necessitating a bottom-up rethinking of attention execution and system architecture.

III. SASDenSebLE Computing Scheme

The proposed SASDenSebLE design fuses the $Q \cdot K^T$ and $A \cdot V$ operations in the attention dataflow, eliminating the need to store the intermediate attention matrix outside the TrPEs. As shown in Fig. 3(b), softmax is merged with $X \cdot V$ (V-MM) and two TrPEs pipeline the execution of sub-MMs of $Q \cdot K^T$ and $X \cdot V$ without stalling. This reduces buffering between two pipeline stages from full attention matrix rows to tiles of size $P \cdot R$ as shown in Fig. 3(a), which fits within the local TrPE buffers. Consequently, SRAM demands become sequence-length independent, improving compute density and resource utilization within the same area budget.

As discussed in Section II-A, buffering full rows of X is driven by 1) max-finding and 2) denominator accumulation, both requiring row-level information. SASDenSebLE addresses this by removing max-finding and deferring normalization (division by the denominator) until after V-MM, enabling tiled $X \cdot V$ execution. Omitting the max-finding

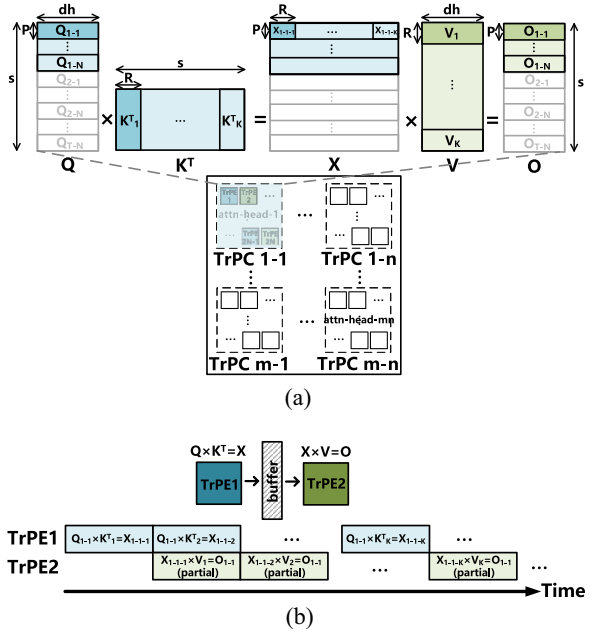


Fig. 3. (a) Mapping of attention module utilizing head parallelism and sequence parallelism. Since the two layers in the attention module are fused, X is used to denote both the attention matrix and the attention score. (b) Pipeline of the dataflow for one attention head with layer fusion of $Q \cdot K$ and $X \cdot V$. For simplicity, the modified softmax operation is excluded.

operation and deferring normalization, however, can precipitate severe overflow of intermediate data values. To mitigate this without resorting to high-bit-width representations, SASDenSebLE introduces three techniques: 1) applying linear approximation to large values in softmax exponentiation, with a threshold to prevent overflow for smaller values; 2) reordering the fusion of modified softmax and V-MM computations to constrain the value range; and 3) applying FP8 precision to small exponentiated values to maintain accuracy. The details of these three techniques are elaborated upon in the subsequent subsections.

A. Saturation-Approximate Softmax

Standard softmax subtracts the max value before exponentiation to prevent overflow. To eliminate this max-finding step, it is crucial to address the potential overflow caused by the high dynamic range inherent in the standard softmax operation. Consequently, SASDenSebLE applies a conditional linear approximation: values below a predefined threshold are exponentiated, while larger values are approximated by a tangent line at the threshold, as illustrated in Fig. 4(a). This approximation scheme is defined in (2), along with the mathematical formulation of saturation-approximate softmax (SA-softmax) in (3), where threshold x_{th} is defined in software considering the data format (Section III-C), $k = \lambda \cdot e^{x_{\text{th}}}$ is the tangent slope of the exponent function at point $(x_{\text{th}}, e^{x_{\text{th}}})$ multiplies a hyperparameter λ, and b can be calculated as $e^{x_{\text{th}}} - k \cdot x_{\text{th}}$. Since the large values are scarce, the linearly approximated values only take up a small proportion

$$\text{SA} - \exp(X_i) = \begin{cases} e^{x_{i,j}}, & x_{i,j} \le x_{\text{th}} \\ k \cdot x_{i,j} + b, & x_{i,j} > x_{\text{th}} \end{cases} \quad j = 1, \ldots, s \tag{2}$$

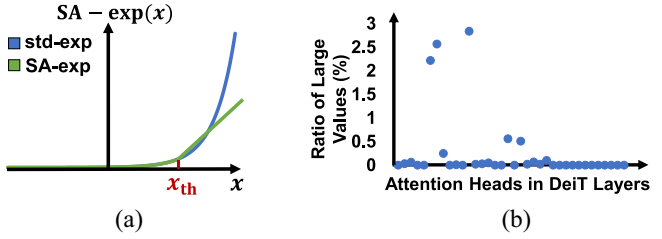


Fig. 4. (a) Illustration of the difference between standard exponential function and SA-exp. (b) Ratio of large values (values that are above x_{th} of all layers in DeiT-Tiny [22]). This ratio of every layer is calculated as the proportion of data points exceeding x_{th} relative to the total number of data points in X.

$$\text{SA} - \text{softmax}(X_i) = \begin{cases} \frac{e^{x_{i,j}}}{\sum \text{SA}-\exp(X_i)}, & x_{i,j} \le x_{\text{th}} \\ \frac{k \cdot x_{i,j} + b}{\sum \text{SA}-\exp(X_i)}, & x_{i,j} > x_{\text{th}} \end{cases} \quad j = 1, \ldots, s. \tag{3}$$

The linear approximation of large values also enables deferring their computation until after V-MM, as linear operations are commutative. Details are provided in the next section.

To assess the feasibility and accuracy impact of this approximation in SA-softmax, DeiT-Tiny [22] is profiled across all layers. In the remainder of this article, sparsity is used to refer to the percentage of attention matrix values exceeding x_{th} (requiring linear approximation). As shown in Fig. 4(b), most layers exhibit zero sparsity, and even the highest sparsity remains below 3%. This indicates that only a small portion of values is affected, helping to preserve model accuracy.

B. Fusing Softmax With Linear Layer

Fig. 5 shows the standard softmax with V-MM and the computation of SASDenSebLE, which fuses the SA-softmax operation with the subsequent V-MM. SASDenSebLE splits into two paths: 1) the X_{dense} datapath for values below x_{th} and 2) the X_{sparse} datapath for values above x_{th} that are marked for linear approximation, along with a binary vector X_{mask} that denotes the positions of elements in X_{sparse}. In the X_{sparse} datapath, $(X_{\text{sparse}} + [b/k]) \cdot V$ is computed before scaling by k to reduce bit-width and avoid overflow. In both paths, normalization (division by the denominator) is deferred until after V-MM and handled by the Post Processing Unit (PPU), breaking the softmax's second temporal dependency. This unblocks the attention pipeline and enables the tiled execution of $Q \cdot K^T$ and $X \cdot V$. The overall operations for the modified softmax, fused with subsequent MM, are expressed in (4)

$$o_{i,j} = \frac{e^{X_{\text{dense}-i}} \cdot V_j + k \cdot \left(X_{\text{sparse}-i} + \frac{b}{k}\right) \cdot V_j}{\sum e^{X_{\text{dense}-i}} + \sum \left(k \cdot X_{\text{sparse}-i} + b\right)}, \; i, j = 1, \ldots, s \tag{4}$$

where

$$x_{\text{dense}-i,j} = \begin{cases} x_{i,j}, & \text{if } x_{i,j} \le x_{\text{th}} \\ -\infty, & \text{otherwise} \end{cases} \tag{5}$$

$$x_{\text{sparse}-i,j} = \begin{cases} x_{i,j}, & \text{if } x_{i,j} > x_{\text{th}} \\ 0, & \text{otherwise} \end{cases}. \tag{6}$$

The rationale for reordering the linear approximation and V-MM in the X_{sparse} datapath is discussed as follows. In the

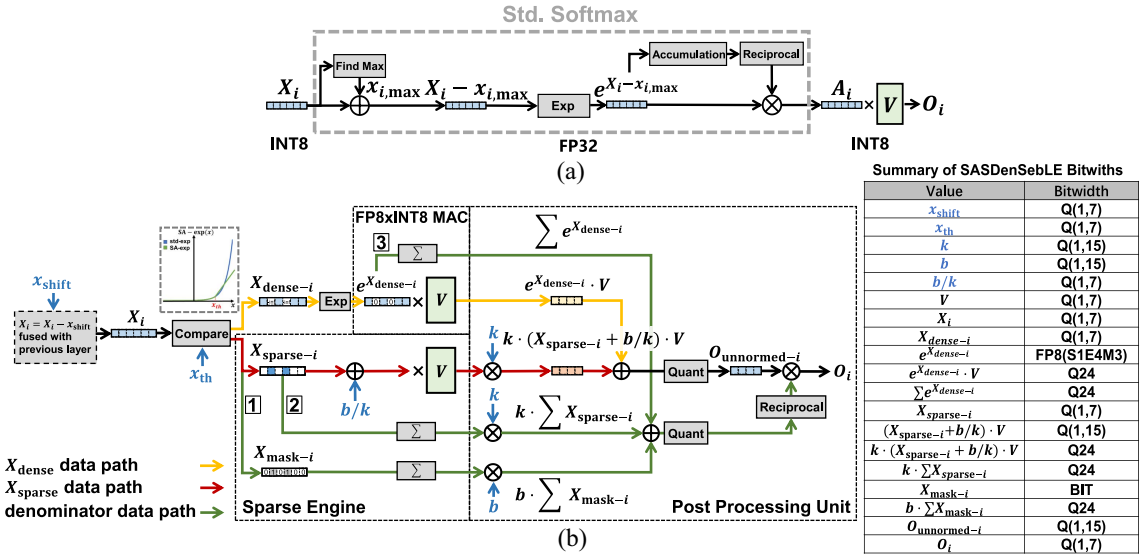


Fig. 5. (a) Computation map of standard softmax and subsequent V-MM. (b) Computation map of SA-softmax and layer fusion with subsequent V-MM. X_i is the result of previous $Q \cdot K^T$ layer, and the bit-width of all intermediate results and parameters are listed on the right side of the graph. For simple illustration, a vector is used instead of a matrix as input.

original order, $X_{\text{linear}} = k \cdot X_{\text{sparse}} + X_{\text{mask}} \cdot b$ is computed before $X_{\text{linear}} \cdot V$, but the large value of k makes X_{linear} exceed the INT16 range, requiring high-bit-width multipliers in the Sparse Engine (Section IV-A2), increasing area and energy costs. SASDenSebLE instead first computes $(X_{\text{sparse}}+[b/k])\cdot V$ without multiplying by k. As shown in Fig. 6(a) and (b), this keeps values within the INT8 range, allowing INT8 compute units. The output of $(X_{\text{sparse}} + [b/k]) \cdot V$ remains within the INT16 range Fig. 6(c) and the final multiplication by k is deferred to the PPU, which already supports high-bit-width operations. This reordering lowers intermediate bit-widths and reduces hardware costs.

Algorithm 2 Calibration Algorithm

Calibration stage:
$x_{\text{P}} \leftarrow \text{quantile}_{\max}(X, P)$
if $x_{\text{P}} > x_{\text{upper}}$ **then**
  $x_{\text{shift}} \leftarrow x_{\text{P}} - x_{\text{upper}}$
  $x_{\text{th}} \leftarrow x_{\text{upper}}$
else
  $x_{\text{shift}} \leftarrow 0$
  $x_{\text{th}} \leftarrow x_{\text{upper}}$
end if
Inference stage:
$X \leftarrow X - x_{\text{shift}}$

C. Threshold Calibration and Precision Selection

Fig. 6(d) demonstrates that a majority of data points in the attention matrix undergoing exponentiation fall within the FP8 region, where FP8 follows the classic S1E4M3 format. Thus, FP8 precision is adequate and presents a favorable tradeoff between accuracy and cost. The threshold x_{th}, which separates exponentiation and linear approximation of values in X, is determined as follows. As Algorithm 2 shows, x_{upper} is denoted as the natural logarithm of the maximum value FP8 can represent. For each attention matrix before executing SA-exp, the lower boundary of the P-maximum of values in X is calculated as x_{P}, where P is a hyperparameter in the range 0.05% to 5%. x_{P} is the initial dense-sparse boundary. However, if x_{P} exceeds x_{upper} (namely, the exponentiation of x_{P} exceeds the upper bound of FP8), the entire attention matrix X is subtracted by a bias x_{shift}, which is calculated as $x_{\text{P}} - x_{\text{upper}}$. The dense-sparse boundary is then set as x_{upper}. This ensures that the exponentiation of the largest dense value can be represented by FP8.

The hyperparameter P is determined through algorithm calibration: P and other quantization-related scaling factors are selected using a calibration dataset via ablation studies and then applied to the test dataset. The choice of P impacts both performance and accuracy. A higher P increases the chance of more large values being linearly approximated (when $x_P > x_{\text{upper}}$), potentially degrading accuracy. Conversely, an extremely small P under conditions where $x_P \gg x_{\text{upper}}$ may lead to underflow, as illustrated in Fig. 6(e), which is also detrimental to accuracy. Additionally, a larger number of large values may prevent the execution latency of the X_{sparse} datapath from being fully hidden behind the X_{dense} datapath in Fig. 5(b), resulting in longer attention pipeline latency. This architectural effect is further discussed in Section IV-A2, and the influence of P variation on accuracy is analyzed in Section V-B.

IV. SASDenSebLE Architecture

Fig. 7 shows the SASDenSebLE architecture and its connection to components, such as the CPU and LPDDR5. The system comprises an array of Transformer-processing clusters (TrPCs) that execute head parallelism in attention layers. Each TrPC contains multiple TrPEs and shared memory, with sequence parallelism applied across TrPE-Pairs.

Each TrPE is designed to support the SA-softmax and layer-fusion execution (right side of Fig. 7). It consists of a storage

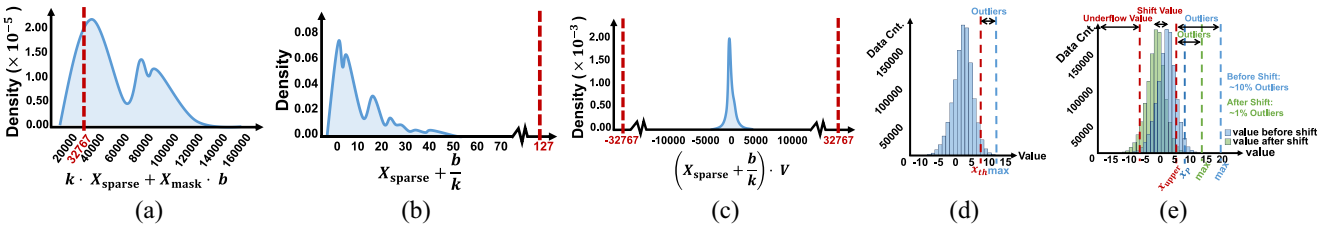


Fig. 6. Data distribution of (a) $k \cdot X_{\text{sparse}} + X_{\text{mask}} \cdot b$, (b) $X_{\text{sparse}} + (b/k)$, and (c) $(X_{\text{sparse}} + [b/k]) \cdot V$. For better visualization, all zero value is excluded. (d) Distribution of X shows that most values fall below the threshold x_{th}. (e) Shifting operation is applied to X when a large number of outliers are present; however, excessive shifting may cause significant underflow. All figures are based on data from DeiT-Tiny [22].

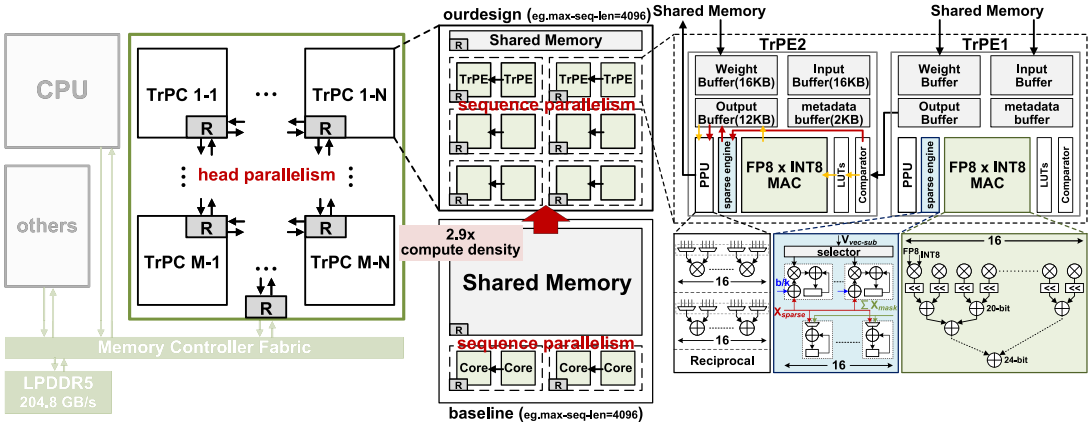


Fig. 7. Overview of SASDenSebLE architecture and micro-architecture.

area (input, weight, and output buffers) and a logic area, which includes: 1) a comparator for thresholding against x_{th}; 2) an exponential LUT; 3) a $FP8 \times INT8$ MAC array; 4) a Sparse Engine with a controller and metadata buffers for sparse operations; and 5) a PPU for merging dense and sparse datapaths and final normalization. TrPEs also flexibly handle FC layers. Two TrPEs form a TrPE-Pair for pipelined attention execution, as illustrated in [Fig. 3(b)].

To enable a fully pipelined attention module and reduce the storage requirements of the attention matrix X, $Q \cdot K^T$ is computed in TrPE1, while the SA-softmax and $X \cdot V$ are performed in TrPE2. The $P \cdot R$-sized tile from TrPE1 is passed to TrPE2, as shown in Fig. 3(a). Data from the output buffer of TrPE1 first passes through the Comparator of TrPE2, then diverges into two datapaths: 1) the dense datapath (yellow in Fig. 7) processes exponentiation via LUTs and an $FP8 \times INT8$ MAC for V multiplication and 2) the sparse path (red in Fig. 7) is handled by the Sparse Engine. Finally, the dual paths converge in the PPU for normalization, and the final result is computed.

A. Building Blocks of Transformer-Processing Element

The following subsections describe the three major components of TrPE, which are specifically designed to support SASDenSebLE dataflow.

1) $FP8 \times INT8$ MAC Array: As discussed in Section III-C, FP8 precision is selected for the values in the dense path after exponentiation. To support this, an $FP8 \times INT8$ MAC is designed, which incorporates minor modifications to a standard MAC array. A shifter is inserted after the 8-bit multiplier to expand the result to the correct value. The adder tree is limited to 24 bits to save area, without compromising model accuracy, as demonstrated in Section V-B. It's important to note that since this is a homogenous system, the MAC array across the entire system is customized to perform $FP8 \times INT8$ dot products, resulting in the quantization of all linear layers in workloads to $FP8 \times INT8$.

2) Sparse Engine: The Sparse Engine is designed to handle the calculation of the sparse datapath. The micro-architecture of the Sparse Engine is shown in the bottom-right part of Fig. 7. It features a 16-wide adder-multiplier-accumulator (AMA) for MM and a 16-wide, 16-bit accumulator (ACC) for denominator calculation. Given that the SASDenSebLE dataflow design ensures the low bit-width of the sparse datapath, the area overhead of the Sparse Engine is minimal.

Fig. 8(a) takes the multiplication of submatrices of X_{sparse} and V as an example to illustrate how Sparse Engine functions and its control flow: For $X_{\text{sparse-sub}}$, all nonzero values are logically pushed to the left (which is done by indexing). At each clock cycle, a column of data is fetched and broadcast it to both AMA and ACC, units with zero-input value are gated for power saving. Values are accumulated in their corresponding row in ACC and the corresponding element of a $V_{\text{vec-sub}}$ is selected for each AMA. For the same $V_{\text{vec-sub}}$, note that it is also used in the $FP8 \times INT8$ MAC Array for dense path calculation of the same X position. The column data of $X_{\text{sparse-sub}}$ is fetched from left to right until meeting the empty column, then the next $V_{\text{vec-sub}}$ is fetched to repeat the same calculation. Since the sparsity of X_{sparse} is large, for most $X_{\text{sparse-sub}}$ blocks, only the first column of its logically pushed counterpart has nonzero values, meaning the calculation of $X_{\text{sparse-sub}} \cdot V_{\text{vec-sub}}$ can be completed at one cycle, as illustrated in Fig. 8(b). Therefore, in most cases, the execution latency

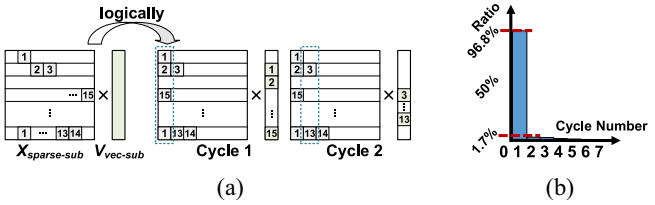


Fig. 8. (a) Sparse Engine dataflow. (b) Execution cycle distribution of X_{sparse} submatrices multiplied with V, derived from DeiT-Tiny with $P = 0.5\%$. For each layer, the head with the highest cumulative cycles is selected for visualization, as head-level parallelism is applied.

of the sparse path is fully hidden by the dense path. The influence of model variation and the choice of hyperparameter P (Section III-C) on execution cycle distribution, and overall performance, is discussed in Section V-C2.

3) Post Processing Unit: The PPU is responsible for merging the dense and sparse datapaths and performing normalization operations. The PPU includes a 16-wide INT16 multiplier, where each input is selected from four data signals, and a 16-wide INT24 adder, with each input similarly selected from four data signals. The denominator passes through the INT8 Reciprocal Unit before being multiplied by the numerator. The PPU is designed to be fully pipelined.

B. Transformer-Processing Cluster (TrPC)

A TrPC consists of an array of TrPE-Pairs and shared memory. Each attention head is mapped to a TrPC for head parallelism, while sequence parallelism is applied across TrPE-Pairs within the TrPC. K and V matrices are loaded from LPDDR5 into shared memory and broadcast to all TrPE-Pairs within the TrPC. As SASDenSebLE stores intermediate attention matrices locally in TrPEs, no further communication with shared memory is needed after broadcasting K and V. For the FC layers, either the input or weight matrix is partitioned across TrPCs according to their respective data amounts, with shared memory broadcasting submatrices to TrPEs. Each TrPE computes its assigned tile independently, with no inter-TrPE or inter-TrPC communication beyond the initial broadcast.

C. Inter-TrPE/Inter-TrPC Communication

As discussed in the previous section, there is no data communication between TrPCs; only broadcasting from the LPDDR5 interface to each TrPC is required. The decision to divide the on-chip shared memory into TrPCs rather than a unified area is driven by the mapping strategy's ability to ensure efficient data reuse within a TrPC while minimizing data communication among TrPCs. This local arrangement of shared memory within TrPCs reduces on-chip routing energy for data transfer. Within a TrPC, broadcasting from shared memory to TrPEs is necessary. For the attention layer to achieve pipelined layer fusion, data communication between the TrPEs in a TrPE-Pair is needed. However, no inter-TrPE communication is required for FC layers. This mapping method ensures minimal cluster-to-cluster and element-to-element communication, which potentially contributes to improved performance.

D. Scalability

The scalability of the entire system is achieved by increasing the number of TrPE-Pairs within a TrPC and the number of TrPCs within the system. This expansion improves both sequence parallelism and head parallelism, resulting in enhanced performance for scenarios involving long sequences and large model sizes. In such cases, segments of the attention matrix can be stored locally within TrPE-Pairs, thus eliminating the need for communication with shared memory or off-chip DRAM during the entire attention execution. This approach facilitates on-chip execution of the attention module with limited on-chip memory requirements.

V. Evaluation

In this section, detailed algorithmic experiments, architecture performance breakdowns, ablation studies, and an overall application demonstration for low-latency edge inference of Transformer NNs are presented.

A. Experimental Setup

1) Models, Datasets, and Evaluation Settings in Algorithm Experiments:

Models and Datasets: The experiments evaluate typical Transformer NNs, including RoBERTa-Base [23] on GLUE [24] for NLP and DeiT [22] and Swin Transformer [25] on ImageNet-1K [26] for CV.

Evaluation Settings in Algorithm Experiments: Standard INT8 quantized models are assessed using I-BERT [27] for NLP and FQ-ViT [8] for CV, with FP32 softmax as the baseline. SASDenSebLE employs hyperparameters $\lambda = 5$ (described in Section III-A) and $P = 1\%$ (Swin and NLP) or 0.5% (DeiT), calibrated as in Section III-C. On top of the standard INT8 Transformer frameworks, $FP8 \times INT8$ quantization of SASDenSebLE is conducted in all linear layers while swapping in the low-bit-precision SA-softmax according to the methodology in Section III. Quantization-aware fine-tuning (QAT) is used for NLP, while post-training quantization (PTQ) suffices for CV, confirming that SASDenSebLE introduces minimal accuracy loss.

2) Hardware Baselines: SASDenSebLE is compared against NVIDIA Jetson AGX Orin [19] an 8 nm industry-standard edge platform for autonomous driving with and without technology scaling [28]. Two conventional DSA configurations and the ViTCoD accelerator [21] are also evaluated. All systems are configured to match the GPU domain area of AGX Orin and assume LPDDR5 DRAM with identical off-chip bandwidth for fair comparison.

GPU: NVIDIA Jetson AGX Orin [19].

Conventional DSAs: Both baselines use the micro-architecture from [13] and online one-pass softmax [14], [17], sharing SASDenSebLE's system architecture and TrPE-level SRAM capacity (Table I). DSA_S provisions TrPC SRAM for the maximum sequence length, while DSA_D scales SRAM to SASDenSebLE's level, exposing DRAM overhead. The differences between the two DSA baselines are simply illustrated in Fig. 9.

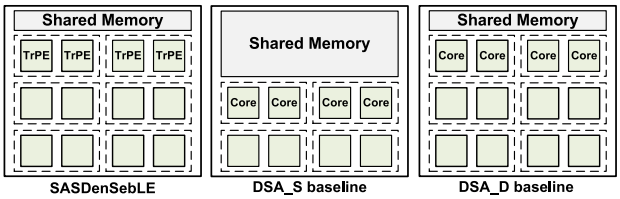


Fig. 9. Illustration of SASDenSebLE, DSA_S baseline, and DSA_D baseline of similar silicon size.

TABLE I
TRPE CONFIGURATION FOR EVALUATION

Comparator	Width	16
LUTs	Depth	256
$FP8 \times INT8$ MAC	Number of MAC lane	16
	Width of MAC lane	16
Sparse Engine	Number of ACC	16
	Number of AMA	16
PPU	Number of INT16 multiplier	16
	Number of INT24 adder	16
	Number of INT8 reciprocal	16
Buffer	Input buffer	16KB
	Weight buffer	16KB
	Output buffer	12KB
	Metadata buffer	2KB
Mapping parameter	P	64
	R	64

TABLE II
AREA BREAKDOWN OF TRPE

Component	Relative Area
SRAM	0.499
$FP8 \times INT8$ MAC	0.297
Sparse Engine	0.051
PPU	0.093
Others	0.060
Total	1.0

ViTCoD: Resources are scaled to match SASDenSebLE's area, including compute and buffering, and account for off-chip activation accesses.

3) Hardware Experiment Setup:

Characteristics: The system occupies 200 mm^2 silicon and runs at 800 MHz. A typical TrPE configuration is shown in Table I with area breakdown in Table II. LPDDR5 (204.8 GB/s) serves as off-chip memory.

Evaluation: All functional modules in SASDenSebLE's TrPE and the baseline cores are implemented in RTL and synthesized using synopsys design compiler with 28 nm CMOS technology. SRAMs use high-density single-port compilers, while the cluster's buffers are configured as 64 KB blocks. The energy consumption of each module was estimated using PrimeTimePX. Layernorm operation across TrPC's is assumed to be executed by software in the edge CPU shown in Fig. 7, which usually takes a small number of cycles. An analytical model-based customized simulator is developed to derive the performance and energy consumption of SASDenSebLE based on post-synthesis data.

B. Algorithm Performance

Tables III and IV present accuracy comparisons for CV and NLP tasks, respectively. With the optimal calibrated hyperparameter P, SASDenSebLE shows an average accuracy drop of 0.51% and 0.61% for DeiT and Swin Transformer in CV tasks after PTQ, and 0.30% for the RoBERTa-Base model in NLP tasks with QAT only. Overall, SASDenSebLE preserves inference accuracy while eliminating high-bit-width softmax computations. Sensitivity analysis of P in CV tasks (Table III) shows that each model favors a specific P, highlighting the importance of careful calibration as discussed in Section III-C.

The robustness to accuracy loss with a calibrated P stems from the statistical distribution of attention values, where only outliers exceeding x_{th} are impacted by approximation. Furthermore, the fusion with V-MM is mathematically equivalent, preventing intermediate quantization noise, while normalization further alleviates absolute computation errors.

C. Performance Analysis

1) Overall Performance: Fig. 10 shows SASDenSebLE's speedup over baselines across sequence lengths from 512 to 8192, while Fig. 11 presents speedup versus core count. A ViT [5] layer with 768 embedding dimension is used for evaluation. The 24-TrPC SASDenSebLE achieves significant gains over GPU due to hardware customization and reduced DRAM access. More importantly, the results demonstrate increasing speedup compared to baseline DSAs with the increase of sequence length, showing weak scaling capability. With total area fixed, experiments in Fig. 11(d) scan through the number of TrPE within a TrPC which exploits sequence parallelism with a total sequence length of 2048. The consistent speedup compared with DSA baselines exhibits strong scaling due to sequence parallelism with reduced attention buffering and memory access.

The DSA_D baseline adopts the same shared-memory and cluster-area configurations as SASDenSebLE, preserving peak MM inference throughput. The baseline core area is 12% smaller than that of TrPE, due to the optimized $INT8 \times INT8$ datapath and removal of the auxiliary Sparse Engine, allowing for slightly more cores within a cluster. However, the costly DRAM spilling of the attention matrix pushes the system to a memory-bound regime, reducing hardware utilization. Experiments show that the SASDensebLE architecture consistently achieves significantly higher speedup over DSA_D. This is specifically demonstrated by the latency breakdown for DSA_D baseline in Figs. 10(a) and 11(a), where the DRAM access latency of attention layers dominates the overall execution latency.

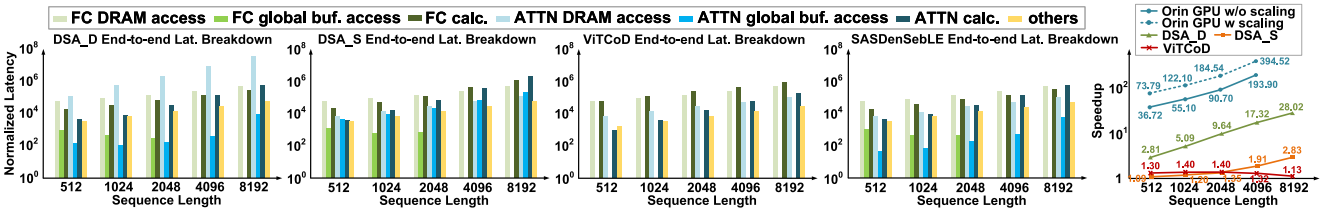
The DSA_S baseline retains all intermediate attention data on-chip but sacrifices compute density due to oversized shared memory, especially at long sequence lengths and high parallelism. This limits the number of Transformer cores compared to SASDenSebLE, resulting in inferior effective compute density. As demonstrated in Figs. 10(b) and 11(b), the calculation latency of both attention and FC layers exceeds that of SASDenSebLE. The comparison with both DSA_D and DSA_S baselines demonstrates that both spilling data to off-chip DRAM and excessive on-chip buffer size degrade the performance, while SASDenSebLE provides an effective solution for achieving low-latency Transformer inference in edge-server applications.

TABLE III
ALGORITHM ACCURACY ON IMAGENET-1K DATASET

Model	DeiT-T	DeiT-S	DeiT-B	DeiT Avg. drop	Swin-T	Swin-S	Swin-B	Swin Avg. drop
FP32	72.21	79.85	81.85	-	81.35	83.20	83.60	-
INT8+FP32 softmax	71.61	79.17	81.20	0.64 (cmp. to FP32)	80.51	82.71	82.97	0.65 (cmp. to FP32)
SASDenSebLE ($P = 0.5\%$)	**70.40**	**78.74**	**81.32**	**0.51 (cmp. to INT8)**	80.39	57.60	58.82	16.46 (cmp. to INT8)
SASDenSebLE ($P = 1\%$)	70.16	78.59	80.18	1.02 (cmp. to INT8)	**79.66**	**82.36**	**82.34**	**0.61 (cmp. to INT8)**
SASDenSebLE ($P = 2\%$)	69.97	78.41	80.96	0.88 (cmp. to INT8)	80.04	82.21	81.38	0.85 (cmp. to INT8)

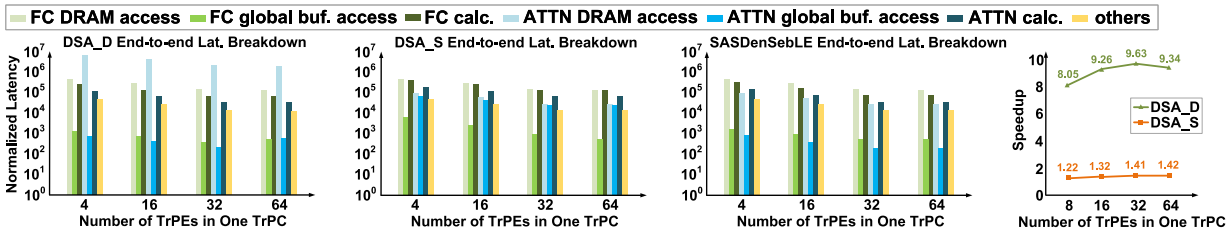
TABLE IV
ALGORITHM ACCURACY ON ROBERTA-BASE MODEL

GLUE Scores	RTE	SST-2	MNLI	QNLI	CoLA	QQP	MRPC	STS-B	Avg. drop
FP32	78.7	94.8	87.8	92.6	83.9	91.4	89.0	90.4	-
INT8+FP32 softmax	76.9	95.2	87.2	92.4	83.4	91.4	89.5	90.5	0.26 (cmp. to FP32)
SASDenSebLE ($P = 1\%$)	77.6	94.4	86.6	92.2	83.1	90.9	89.5	89.8	0.30 (cmp. to INT8)



(a) (b) (c) (d) (e)

Fig. 10. Normalized end-to-end latency breakdown for executing the ViT layer with a 768-dimensional embedding at varying sequence lengths: (a) DSA_D baseline, (b) DSA_S baseline, (c) ViTCoD accelerator, and (d) SASDenSebLE. Note that the vertical coordinate is a logarithmic coordinate. (e) Speedup of the SASDenSebLE architecture over the four baselines at different sequence lengths. No GPU data is available for a sequence length of 8192, as the memory demand exceeds the GPU's upper runnable limitation. *No FC global buffer access latency is observed in some cases of (a), (b), and (d) due to latency hiding. ViTCoD does not model a global buffer, so no FC/ATTN global buffer access appears in (d).



(a) (b) (c) (d)

Fig. 11. Normalized end-to-end latency breakdown for executing the ViT layer with a 768-dimensional embedding when scaling up the hardware resources: (a) DSA_D baseline, (b) DSA_S baseline, and (c) SASDenSebLE. Note that the vertical coordinate is a logarithmic coordinate. (d) Speedup of the SASDenSebLE architecture over the two baselines at different TrPE numbers. *GPU and ViTCoD baselines are excluded from Fig. 11 due to fixed GPU resources and undisclosure area details in ViTCoD.

ViTCoD reduces attention overhead via sparsity, but as Fig. 10(c) shows, its FC layer latency exceeds SASDenSebLE's. This is due to fewer compute resources under the same area budget, as sparsity benefits attention but not dense FC layers, resulting in higher FC computation cycles.

2) Impact of Hyperparameter and Model Variation: As discussed in Section IV-A2, both model variation and changes in hyperparameter P influence the sparsity of large values the attention matrix, occasionally causing the X_{sparse} datapath to require more cycles than the X_{dense} datapath, thus affecting execution performance. However, this impact is minor. Table V presents evaluations of three DeiT models under three different P values, using systems with 24 TrPC and a workload sequence length of 4096. The ≤ 1 Cycle Percentage denotes the proportion of subblocks in the X_{sparse} datapath that complete within 1 Cycle (0 or 1 cycle; 0 indicates no nonzero values), while Avg. Cycles indicates the average execution cycles per subblock across all layers. The execution of subblocks completing within 1 cycle can be fully hidden behind the X_{sparse} datapath. Due to the head-parallelism adopted in SASDenSebLE, the head with the highest average execution cycles per layer is selected to represent the X_{sparse} datapath overhead. The speedups over DSA_S and DSA_D baselines indicate that larger models yield smaller gains, as the attention layer accounts for a reduced portion of overall execution time (ATTN./Overall in Table V), where SASDenSebLE provides the most benefit. Additionally, larger models and higher P values increase the average cycles per subblock in the X_{sparse} datapath, resulting in mild performance degradation. However, this overhead is limited and does not significantly impact overall system performance.

TABLE V
SPEEDUP W.R.T. P AND MODEL VARIATION

Model	$P(\%)$	Speedup (DSA_S, DSA_D)	ATTN./Overall	≤ 1 Cycle Percentage (%)	Avg. Cycles
DeiT-T	0.5	(2.82, 39.53)	0.73	96.79	1.06
	1	(2.76, 38.81)	0.74	95.17	1.09
	2	(2.71, 38,01)	0.74	93.54	1.14
DeiT-S	0.5	(2.46, 29.33)	0.55	95.27	1.07
	1	(2.38, 28.33)	0.56	90.68	1.17
	2	(2.31, 27.5)	0.57	89.05	1.25
DeiT-B	0.5	(2.09, 16.16)	0.35	85.90	1.27
	1	(2.01, 15.60)	0.37	80.62	1.45
	2	(1.93, 14.95)	0.40	75.24	1.67

3) Take PIM Into Consideration: Process-in-memory (PIM) architectures enhance SRAM with compute capabilities to achieve higher compute density [29], [30], [31]. This approach is orthogonal to the proposed method in this article, as the on-chip shared memory can be upgraded to PIM to further improve compute density, albeit with additional area overhead. An analytical-level evaluation is performed by applying PIM to the shared memory of TrPC to accelerate FC layer computations in both SASDenSebLE and the DSA_S baseline, where DSA_S is configured with larger on-chip shared memory and lower compute density. The shared memory is modified to PIM with a storage density of 2.4 Mb/mm^2 and an area efficiency of 16.7 TOPS/mm^2 at 1100 MHz [32], while other experimental settings are consistent with those in Fig. 10.

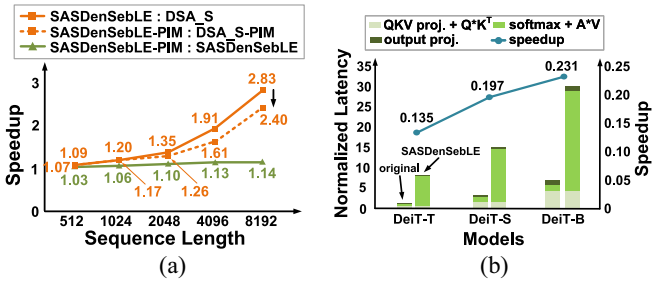
As shown in Fig. 12(a), the speedup of SASDenSebLE-PIM over SASDenSebLE is limited, as SASDenSebLE already achieves high compute density, and the calculation cycles of FC layers contribute minimally to overall latency. The speedup of SASDenSebLE-PIM over DSA_S-PIM diminishes as sequence length increases, compared to the original speedup without PIM, since DSA_S allocates more area to shared memory in longer sequence scenarios, resulting in higher compute density when PIM is applied. However, SASDenSebLE-PIM still outperforms DSA_S-PIM significantly, as PIM only accelerates FC layers but does not mitigate intrarow dependencies in softmax, which remain a bottleneck for fine-grained attention pipelining. Furthermore, PIM does not alleviate off-chip DRAM communication overhead.

D. Ablation Study

To investigate the necessity of linear approximation for values exceeding the threshold and the importance of hardware-software co-design, two ablation studies are conducted. First, the SASDenSebLE software is implemented without applying linear approximation for large values. Instead, values exceeding the largest representable FP8 value after exponentiation in the softmax operation were clamped. Specifically, the X_{sparse} datapath, along with denominator datapaths 1 and 2 in Fig. 5(b), were removed, and the bitwidths of the remaining datapaths followed those listed in Fig. 5(b). This software-only ablation study was conducted on an NVIDIA GeForce RTX 2080 Ti GPU, with workloads involving a sequence length of 196. As shown in Table VI, directly removing the max subtraction in the softmax without addressing overflow values caused a significant accuracy drop, even in small models. This demonstrates the necessity of applying linear approximation for large values in SASDenSebLE compute scheme.

TABLE VI
ALGORITHM ACCURACY DROP ON DEIT MODEL

Model	DeiT-T	DeiT-S	DeiT-B
SASDenSebLE ($P = 0.5\%$)	70.40	78.74	81.32
SASDenSebLE w/o linear approx.	67.02	34.88	1.64



(a) (b)

Fig. 12. (a) Speedup variation when applying PIM to shared memory in TrPC. (b) Normalized attention latency comparison between the DeiT with standard softmax and the SASDenSebLE software-only implementation, along with the speedup of SASDenSebLE over the original DeiT on an NVIDIA GeForce RTX 2080 Ti GPU.

To demonstrate the necessity of SASDenSebLE DSA design, the performance of the attention layer (including the Q, K, V projections and the output projection) of a PyTorch-based SASDenSebLE compute scheme implementation within DeiT is evaluated on an NVIDIA GeForce RTX 2080 Ti GPU, alongside the original PyTorch implementation of DeiT. However, this implementation of SASDenSebLE compute scheme on GPU is not entirely faithful, as GPU lacks specialized hardware, such as FP8-multiplying INT8 units and dense/sparse datapaths, and custom control and data flow, which misaligns with the proposed design.

Measured using FasterTransformer [33], Fig. 12(b) presents the normalized latency and speedup across three DeiT models. The SASDenSebLE software on GPU exhibits higher execution time, mainly due to extensive data format conversions and diverging datapaths that are not GPU-optimized in softmax $+ A \times V$ computations. This suggests that SASDenSebLE compute scheme cannot be directly applied to GPUs for two reasons: first, it requires flexible data bitwidths (ranging from 8-bit to 24-bit fixed-point) and specialized hardware support, including sparse engines, FP8-multiplying INT8 MAC units, and dense/sparse datapaths, which GPUs lack. Second, it relies on a dedicated control flow for pipelined attention layer fusion and latency hiding between sparse and dense datapaths, which is difficult to manage directly on an GPU. This underscores the necessity of a DSA design. Section VI-A discusses other works aimed at reducing the data storage requirements of the attention matrix on GPU platforms.

E. Energy Efficiency

Fig. 13 compares the energy consumption of the Orin GPU, the baseline DSAs, and the SASDenSebLE system under the same settings as the speedup experiments shown in Fig. 10.

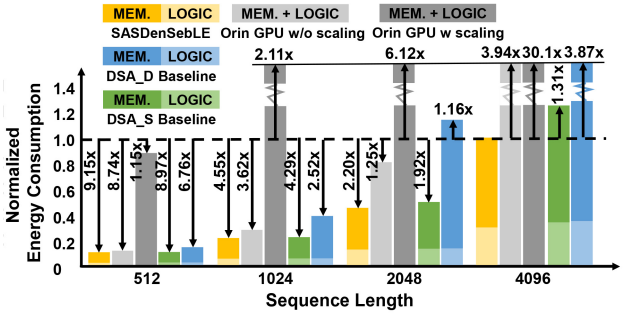


Fig. 13. Energy consumption of a Transformer layer over GPU (with and without technology scaling) and two DSA baselines as sequence length grows. *Off-chip memory access energy and logic energy within the GPU are not separated, as the profiling tool reports only the total GPU power.

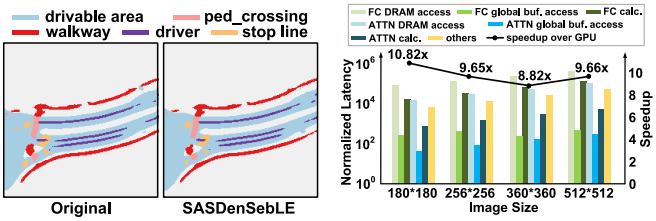


Fig. 14. (a) Segmentation results of the BEV map using the original algorithm and the SASDenSebLE compute scheme. (b) Latency breakdown of Swin-Tiny within BEVFusion running on the SASDenSebLE architecture, along with the speedup over the NVIDIA Jetson AGX Orin GPU.

SASDenSebLE outperforms the GPU in energy efficiency, especially with longer sequence lengths.

The comparison with the DSA_S baseline highlights the benefit of layer-fusion execution, which reduces local high-bit-width intermediate data access in the standard softmax. The significant energy reduction compared to the DSA_D baseline is primarily due to a decrease in DRAM access. The energy-efficiency gain also scales with sequence length, achieving 1.31× and 3.87× improvements over the DSA_S and DSA_D baselines, respectively, at a sequence length of 4096.

F. End-to-End Demonstration

The recent BEVFusion [2], [20] algorithm is selected for the end-to-end demonstration. The SASDenSebLE compute scheme is implemented in the Transformer backbone, and the mIoU dropped from 62.95% for the full precision model to 62.91%, a mere 0.04% decrease. Fig. 14(a) presents the BEV map segmentation results using the original BEVFusion and the modified version with the SASDenSebLE compute scheme. The visual similarity between the two indicates that SASDenSebLE preserves a high level of model accuracy.

We showcase the speedup achieved by SASDenSebLE over the NVIDIA Jetson AGX Orin GPU through the end-to-end execution of the Swin-Tiny [25] model, which forms the backbone of BEVFusion algorithm, using a window size of 7 × 7. The mapping of the FC layers in Swin-Tiny follows the method described in Section IV-B. For the attention layers, due to Swin Transformer's unique window mechanism, self-attention within each window is assigned to individual TrPE pairs, enabling window-level parallelism across TrPEs, while head-level parallelism is maintained across TrPCs. Fig. 14(b) shows the end-to-end latency breakdown of Swin-Tiny and the corresponding speedup over the NVIDIA Jetson AGX Orin GPU, illustrating that the speedup can reach up to 10.82×, highlighting the potential for faster inference speeds with high-resolution images. It is worth noticing that the Swin Transformer model is designed to limit the effective sequence length in the dataflow by window partitioning, making it hardware-friendly to conventional GPU architectures. Thus, it is a conservative end-to-end demonstration, while a greater speedup approaching number in Section V-C1 can be achieved with other Transformer NNs widely used in autonomous-driving cases.

VI. Related Works and Discussions

In recent years, various works have aimed to accelerate Transformer layers via algorithmic, architectural, and circuit-level innovations. These include quantization [8], [34], sparsity exploitation [9], [12], nonlinear kernel optimization [14], [15], [16], and end-to-end accelerator design [13]. There have been pioneer works within this landscape, yet with notable distinctions, they will be discussed in the following sections.

A. Efficient Attention Algorithm

Recent works on GPU and FPGA platforms have introduced efficient kernel- and scheduling-level modifications to the attention module for acceleration.

FlashAttention [18] decouples temporal dependencies in softmax, enabling sequence parallelism and reducing GPU memory accesses via online max tracking and progressive recomputation of softmax denominator. However, it incurs recomputation overhead, relies on high-bit-width operations, and is not directly adaptable to domain-specific architectures. FlashDecoding++ [35] further optimizes FlashAttention by introducing a unified max to reduce data updates and recomputing only on overflow. However, it lacks software-hardware co-design and retains full-precision (FP32) softmax, whereas this work enables low-bit-width implementations.

Edge-MoE [17], an FPGA-based work, reduces softmax complexity by transforming the standard three-pass computation into a single-pass online algorithm, thereby lowering compute latency and memory overhead. While Edge-MoE employs an accurate softmax without compromising model quality, SASDenSebLE introduces approximations on large values to trade minor accuracy loss for performance gains. These two approaches could be complementary in larger models: accuracy-sensitive layers could leverage Edge-MoE's precise softmax, while SASDenSebLE could be applied to other layers for higher efficiency. Additional control logic would be necessary to support the integration of both methods.

B. Efficient Transformer Accelerator

Numerous works focus on accelerating the Transformer attention module [9], [36], but often overlook end-to-end

execution or lack scalability and parallelism discussions [12], [37], [38]. Work [13] explores end-to-end execution and system scalability but prioritizes quantization support, leaving attention dataflow unoptimized and causing significant intermediate storage and data movement overhead, especially for long sequences. In contrast, this work reduces intermediate attention matrix data via a redesigned attention dataflow. DOTA [10] also addresses token parallelism, but limits it within a single core, restricting scalability. The proposed work adopts sequence parallelism across the core array to enable strong scaling.

This work extends our previous conference publication [39] by providing a deeper analysis of algorithmic approximations, precision selection, calibration, and micro-architecture design. It also includes additional evaluations, such as comparisons with GPUs, ViTCoD, and PIM, a detailed latency breakdown, ablation studies, and a real-world autonomous driving case demonstrating speedup over GPU.

C. Limitations and Future Works

While SASDenSebLE demonstrates significant performance and energy efficiency gains, several limitations and opportunities for further enhancement remain. Specifically, SASDenSebLE may face challenges when applied to significantly larger ViTs or large language models (LLMs). In LLMs, attention matrices typically contain more large outliers than ViTs, which could degrade model quality if addressed solely with simple linear approximations in softmax. Specialized techniques will be necessary to handle these outliers when extending the method to LLMs. Additionally, hyperparameter tuning may differ across ViT models, requiring thorough profiling for optimal settings.

Future work could focus on developing adaptive approximation methods that adjust dynamically to the distribution of attention values, improving robustness across Transformer architectures. Extending SASDenSebLE to broader applications, such as generative models and multimodal systems, also presents an exciting research direction.

VII. Conclusion

This work presents SASDenSebLE, an efficient Transformer inference architecture for edge devices, featuring hardware-software co-optimization across algorithm, micro-architecture, and system levels. A saturation-linear softmax approximation enables low-bit-width, layer-fused self-attention execution. A dense-sparse-hybrid datapath and a parallelism-insensitive system design ensure strong scalability and high compute density. SASDenSebLE delivers up to 2.83× and 28.02× speedups, with 1.31× and 3.87× energy efficiency gains over two DSA baselines, and a 1.40× speedup over a state-of-the-art ViT accelerator.

References

[1] Z. Li et al., "BEVFormer: Learning bird's-eye-view representation from multi-camera images via spatiotemporal transformers," in *Proc. Eur. Conf. Comput. Vis.*, 2022, pp. 1–18.

[2] T. Liang et al., "BEVFusion: A simple and robust LiDAR-camera fusion framework," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 35, 2022, pp. 10421–10434.

[3] J. Devlin et al., "BERT: Pre-training of deep bidirectional transformers for language understanding," 2018, *arXiv:1810.04805*.

[4] H. Touvron et al., "LLaMA: Open and efficient foundation language models," 2023, *arXiv:2302.13971*.

[5] A. Dosovitskiy et al., "An image is worth 16x16 words: Transformers for image recognition at scale," 2020, *arXiv:2010.11929*.

[6] H. Yin et al., "A-ViT: Adaptive tokens for efficient vision transformer," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2022, pp. 10809–10818.

[7] Y. Rao, Wenliang Zhao, Benlin Liu, Jiwen Lu, J. Zhou, and C.-J. Hsieh, "DynamicViT: Efficient vision transformers with dynamic token sparsification," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 34, 2021, pp. 13937–13949.

[8] Y. Lin, T. Zhang, P. Sun, Z. Li, and S. Zhou, "FQ-ViT: Post-training quantization for fully quantized vision transformer," 2021, *arXiv:2111.13824*.

[9] H. Wang, Z. Zhang, and S. Han, "SpAtten: Efficient sparse attention architecture with cascade token and head pruning," in *Proc. IEEE Int. Symp. High-Perform. Comput. Archit. (HPCA)*, 2021, pp. 97–110.

[10] Z. Qu, L. Liu, F. Tu, Z. Chen, Y. Ding, and Y. Xie, "DOTA: Detect and omit weak attentions for scalable transformer acceleration," in *Proc. 27th ACM Int. Conf. Archit. Support Program. Lang. Oper. Syst.*, 2022, pp. 14–26.

[11] T. J. Ham et al., "ELSA: Hardware-software co-design for efficient, lightweight self-attention mechanism in neural networks," in *Proc. ACM/IEEE 48th Annu. Int. Symp. Comput. Archit. (ISCA)*, 2021, pp. 692–705.

[12] T. Tambe, J. Zhang, C. Hooper, T. Jia, P. N. Whatmough, and J. Zuckerman, "22.9 a 12nm 18.1 TFLOPs/W sparse transformer processor with entropy-based early exit, mixed-precision predication and fine-grained power management," in *Proc. IEEE Int. Solid-State Circuits Conf. (ISSCC)*, 2023, pp. 342–344.

[13] B. Keller et al., "A 95.6-TOPS/W deep learning inference accelerator with per-vector scaled 4-bit quantization in 5 nm," *IEEE J. Solid-State Circuits*, vol. 58, no. 4, pp. 1129–1141, Apr. 2023.

[14] J. R. Stevens, R. Venkatesan, S. Dai, B. Khailany, and A. Raghunathan, "Softermax: Hardware/software co-design of an efficient softmax for transformers," in *Proc. 58th ACM/IEEE Design Autom. Conf. (DAC)*, 2021, pp. 469–474.

[15] Y. Gao, W. Liu, and F. Lombardi, "Design and implementation of an approximate softmax layer for deep neural networks," in *Proc. IEEE Int. Symp. Circuits Syst. (ISCAS)*, 2020, pp. 1–5.

[16] W. Wang, S. Zhou, W. Sun, P. Sun, and Y. Liu, "SOLE: Hardware-software co-design of softmax and LayerNorm for efficient transformer inference," in *Proc. IEEE/ACM Int. Conf. Comput. Aided Design (ICCAD)*, 2023, pp. 1–9.

[17] R. Sarkar, H. Liang, Z. Fan, Z. Wang, and C. Hao, "Edge-MoE: Memory-efficient multi-task vision transformer architecture with task-level sparsity via mixture-of-experts," in *Proc. IEEE/ACM Int. Conf. Comput. Aided Design (ICCAD)*, 2023, pp. 1–9.

[18] T. Dao, D. Fu, S. Ermon, A. Rudra, and C. Ré, "FlashAttention: Fast and memory-efficient exact attention with IO-awareness," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 35, 2022, pp. 16344–16359.

[19] "NVIDIA Jetson Orin AGX." NVIDIA. 2021. [Online]. Available: https://www.nvidia.com/content/dam/en-zz/Solutions/gtcf21/jetson-orin/nvidia-jetson-agx-orin-technical-brief.pdf

[20] Z. Liu et al., "BEVFusion: Multi-task multi-sensor fusion with unified bird's-eye view representation," in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, 2023, pp. 2774–2781.

[21] H. You et al., "ViTCoD: Vision transformer acceleration via dedicated algorithm and accelerator co-design," in *Proc. IEEE Int. Symp. High-Perform. Comput. Archit. (HPCA)*, 2023, pp. 273–286.

[22] H. Touvron, M. Cord, M. Douze, F. Massa, A. Sablayrolles, and H. Jegou, "Training data-efficient image transformers & distillation through attention," in *Proc. Int. Conf. Mach. Learn.*, 2021, pp. 10347–10357.

[23] Y. Liu et al., "RoBERTa: A robustly optimized bert pretraining approach," 2019, *arXiv:1907.11692*.

[24] A. Wang, A. Singh, J. Michael, F. Hill, O. Levy, and S. R. Bowman, "GLUE: A multi-task benchmark and analysis platform for natural language understanding," 2018, *arXiv:1804.07461*.

[25] Z. Liu et al., "Swin transformer: Hierarchical vision transformer using shifted windows," in *Proc. IEEE/CVF Int. Conf. Comput. Vis.*, 2021, pp. 10012–10022.

[26] J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li, and L. Fei-Fei, "ImageNet: A large-scale hierarchical image database," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2009, pp. 248–255.
[27] S. Kim, A. Gholami, Z. Yao, M. W. Mahoney, and K. Keutzer, "I-BERT: Integer-only BERT quantization," in *Proc. Int. Conf. Mach. Learn.*, 2021, pp. 5506–5518.
[28] A. Stillmaker and B. M. Baas, "Scaling equations for the accurate prediction of CMOS device performance from 180 nm to 7 nm," *Integration*, vol. 58, pp. 74–81, Jun. 2017. [Online]. Available: https://api.semanticscholar.org/CorpusID:205666078
[29] Y. Ding, C. Liu, M. Duan, W. Chang, K. Li, and K. Li, "HAIMA: A hybrid SRAM and DRAM accelerator-in-memory architecture for transformer," in *Proc. 60th ACM/IEEE Design Autom. Conf. (DAC)*, 2023, pp. 1–6.
[30] F. Tu et al., "ReDCIM: Reconfigurable digital computing- in -memory processor with unified FP/INT pipeline for cloud AI acceleration," *IEEE J. Solid-State Circuits*, vol. 58, no. 1, pp. 243–255, Jan. 2023.
[31] M. Zhou, W. Xu, J. Kang, and T. Rosing, "TransPIM: A memory-based acceleration via software-hardware co-design for transformer," in *Proc. IEEE Int. Symp. High-Perform. Comput. Archit. (HPCA)*, 2022, pp. 1071–1085.
[32] Y. He et al., "34.7 A 28nm 2.4Mb/mm2 6.9—16.3TOPS/mm2 eDRAM-LUT-based digital-computing-in-memory macro with in-memory encoding and refreshing," in *Proc. IEEE Int. Solid-State Circuits Conf. (ISSCC)*, vol. 67, 2024, pp. 578–580.
[33] "FasterTransformer." NVIDIA. 2022. [Online]. Available: https://github.com/NVIDIA/FasterTransformer
[34] G. Xiao, J. Lin, M. Seznec, H. Wu, J. Demouth, and S. Han, "SmoothQuant: Accurate and efficient post-training quantization for large language models," 2024, *arXiv:2211.10438*.
[35] K. Hong et al., "Flashdecoding++: Faster large language model inference on GPUs," 2023, *arXiv:2311.01282*.
[36] T. J. Ham et al., "A3: Accelerating attention mechanisms in neural networks with approximation," in *Proc. IEEE Int. Symp. High Perform. Comput. Archit. (HPCA)*, 2020, pp. 328–341.
[37] Y. Wang et al., "A 28nm 27.5TOPS/W approximate-computing-based transformer processor with asymptotic sparsity speculating and out-of-order computing," in *Proc. IEEE Int. Solid-State Circuits Conf. (ISSCC)*, vol. 65, 2022, pp. 1–3.
[38] B. Keller et al., "A 17–95.6 TOPS/W deep learning inference accelerator with per-vector scaled 4-bit quantization for transformers in 5nm," in *Proc. IEEE Symp. VLSI Technol. Circuits (VLSI Technol. Circuits)*, 2022, pp. 16–17.
[39] L. He et al., "Exploring approximation and dataflow co-optimization for scalable transformer inference architecture on the edge," in *Proc. IEEE 37th Int. Syst. Chip Conf. (SOCC)*, 2024, pp. 1–6. [Online]. Available: https://api.semanticscholar.org/CorpusID:273852402

Liu He (Graduate Student Member, IEEE) received the B.S. degree in electronic engineering from Tsinghua University, Beijing, China, in 2023, where she is currently pursuing the Ph.D. degree with the Department of Electronic Engineering.

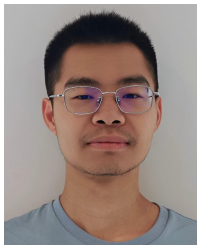
Her current research interests include efficient domain-specific architecture and data flow design.

Yujin Wang (Graduate Student Member, IEEE) received the B.S. degree from the Department of Electronic Engineering, Tsinghua University, Beijing, China, in 2024, where he is currently pursuing the Ph.D. degree.

His research interests include neural network compression and hardware–software co-design.

Zongle Huang (Graduate Student Member, IEEE) received the B.S. degree from the Department of Electronic Engineering, Tsinghua University, Beijing, China, in 2023, where she is currently pursuing the Ph.D. degree.

Her research interests include distributed systems and domain-specific accelerator design.



Shupei Fan (Graduate Student Member, IEEE) received the B.S. degree from Tsinghua University, in 2019, where he is currently pursuing the Ph.D. degree.

His current research interests include architecture designs on CNN and transformer accelerators.

Chen Tang (Graduate Student Member, IEEE) received the B.S. degree from the Department of Electronic Engineering, Tsinghua University, Beijing, China, in 2020, where he is currently pursuing the Ph.D. degree.

His research interests include neural network compression and machine learning accelerators.

Shuyuan Zhang (Graduate Student Member, IEEE) received the B.S. degree in electronic engineering from Tsinghua University, Beijing, China, in 2023, where he is currently pursuing the Ph.D. degree with the Department of Electronic Engineering.

His current research interests include AI processors and digital VLSI architecture.

Luchang Lei (Graduate Student Member, IEEE) received the B.S. degree in electronic science and technology from Tsinghua University, Beijing, China, in 2022, where he is currently pursing the Ph.D. degree in electronic science and technology with the Department of Electronic Engineering.

His research interests include in-memory computing architecture and circuit, and hardware acceleration of homomorphic encryption.

Huazhong Yang (Fellow, IEEE) received the B.S. degree in microelectronics and the M.S. and Ph.D. degrees in electronic engineering from Tsinghua University, Beijing, China, in 1989, 1993, and 1998, respectively.

In 1993, he joined the Department of Electronic Engineering, Tsinghua University, where he has been a Full Professor since 1998. He has been in charge of several projects, including projects sponsored by the National Science and Technology Major Project, the 863 Program, the NSFC, and several international research projects. He has authored or co-authored over 500 technical articles, seven books, and over 180 granted Chinese patents. His research interests include wireless sensor networks, data converters, energy-harvesting circuits, nonvolatile processors, and brain-inspired computing.

Prof. Yang received the Distinguished Young Researcher by NSFC in 2000, the Cheung Kong Scholar by the Chinese Ministry of Education (CME) in 2012, the Science and Technology Award First Prize by the China Highway and Transportation Society in 2016, the Technological Invention Award First Prize by CME in 2019, the Gold Prize of iNEA 2019, and several Best Paper Awards, including ISVLSI 2012, FPGA 2017, NVMSA 2017, and ASP-DAC 2019. He has served as a Navigating Committee Member for AsianHOST 2018 and the TPC Member for ASP-DAC 2005, APCCAS 2006, ICCCAS 2007, ASQED 2009, and ICGCS 2010. He has served as the Chair for the Northern China ACM SIGDA Chapter Science in 2014 and the General Co-Chair for ASP-DAC 2020.

Yongpan Liu (Senior Member, IEEE) received the B.S., M.S., and Ph.D. degrees from Tsinghua University, Beijing, China, in 1999, 2002, and 2007, respectively.

He is currently a Full Professor (Cheung Kong Scholar) with the Department of Electronic Engineering, Tsinghua University.

Prof. Liu has received the Under 40 Young Innovators Award DAC 2017, the Best Paper/Poster Award from ASPDAC 2021, 2017, the Micro Top Pick 2016, the HPCA 2015, and the Design Contest Awards of ISLPED in 2012, 2013, and 2019. He served as the General Secretary for ASP-DAC 2021 and the Technical Program Chair for ICAC2023. He is a Program Committee Member for ISSCC, A-SSCC, and CICC. He was an Associate Editor of the IEEE TRANSACTIONS ON COMPUTER–AIDED DESIGN OF INTEGRATED CIRCUITS AND SYSTEMS, IEEE TRANSACTIONS ON CIRCUITS AND SYSTEMS—PART II: EXPRESS BRIEFS, and *IET Cyber–Physical Systems*. He also served as an A-SSCC2020/AICAS2022 Tutorial Speaker and an IEEE CASS Distinguished Lecturer 2021.

Hongyang Jia (Member, IEEE) received the B.E. degree from Tsinghua University, Beijing, China, in 2014, and the M.A. and Ph.D. degrees from Princeton University, Princeton, NJ, USA, in 2016 and 2021, respectively.

From 2021 to 2022, he was a Postdoctoral Researcher with NVIDIA, Santa Clara, CA, USA. Since August 2022, he has been with the Department of Electronic Engineering, Tsinghua University, where he is currently an Assistant Professor. His research focuses on advanced computing technologies and systems, rethinking how emerging computing fabrics can be integrated with architectures and algorithms to enhance system efficiency and performance. His primary research interests include approximate computing, mixed-signal computing, in-memory computing, energy-efficient circuits, and architectures for robotic computing and security applications.

Prof. Jia received the 2017 Outstanding Student Designer Award from ADI Inc., and the 2022 Bede Liu Best Dissertation Award in Electrical and Computer Engineering from Princeton ECE. He serves for the technical program committees for the IEEE/ACM ICCAD Conference.